🐾 Proyecto: Tamagotchi en Python (sin POO)

🎯 Objetivo general

Desarrollar una aplicación interactiva en Python que simule el comportamiento de una mascota virtual (Tamagotchi) ejecutándose en terminal, sin utilizar programación orientada a objetos.

El proyecto deberá estar estructurado en múltiples módulos funcionales y demostrar el uso de conceptos fundamentales de programación, incluyendo estructuras de datos (diccionarios y listas), funciones, control de flujo y gestión de archivos.

Además, la aplicación deberá:

- gestionar el estado dinámico de la mascota a lo largo del tiempo
- permitir la interacción del usuario mediante un menú en terminal
- utilizar archivos JSON para la persistencia de datos
- mostrar información visual mediante ASCII art
- integrar eventos y mecánicas que influyan en la evolución del juego

El objetivo es construir un programa completo, organizado y mantenible que refleje buenas prácticas de desarrollo en Python sin recurrir a la programación orientada a objetos.

📦 Estructura esperada

```
tamagotchi/
│
├── main.py
├── game_logic.py
├── display.py
├── storage.py
├── events.py
├── utils.py
│
├── data/
│   ├── mascotas.json
│   ├── items.json
│   └── saves/
│
├── ascii/
│   ├── feliz.txt
│   ├── triste.txt
│   └── muerto.txt
│
└── README.md
```

Modalidades de entrega

El proyecto se realiza mediante un sistema de **entregas progresivas**, donde cada fase debe completarse en orden.

Funcionamiento general

El proyecto está dividido en varias entregas (fases):

1. El estudiante solo puede acceder a la **primera entrega** al inicio.
2. Para desbloquear la siguiente entrega:
    - debe haber enviado la anterior
    - el profesor debe haberla corregido
    - el profesor debe haberla **liberado**
3. Sin validación del profesor, el proyecto **no avanza**.

Proceso de entrega

Para cada fase:

- el estudiante debe subir un archivo `.zip`
- el archivo debe contener el proyecto correspondiente a esa fase
- el nombre del archivo es libre, pero debe ser claro.

Ejemplo: JuanPablo_fase_1.zip

Contenido del ZIP

Cada entrega debe contener:

- código fuente (`.py`)
- archivos JSON si se utilizan
- carpeta `ascii/` si aplica
- README si es requerido

⚠ No subir:

- entornos virtuales (`venv`)
- archivos innecesarios
- ejecutables

🔁 Reentrega

Un estudiante puede volver a enviar una entrega:

- la entrega anterior se reemplaza
- la nota y comentarios se reinician
- la fase vuelve a estado **pendiente de corrección**
- la liberación se cancela

👩‍🏫 Corrección del profesor

Para cada entrega, el profesor puede:

- asignar una nota
- escribir un comentario
- decidir si la entrega está validada

Estados posibles:

- 📦 Entregada
- 📝 Corregida
- ☑️ Corregida y liberada

🔓 Sistema de desbloqueo

Una entrega se desbloquea solo si:

- la entrega anterior está:
  - corregida
  - y **liberada**

Si no:

> 🔒 La entrega permanece bloqueada

⏱️ Fechas límite

Cada entrega puede tener una fecha límite:

- si se supera:
  - no se puede entregar (según configuración)
  - o se penaliza la nota

⚠ Errores comunes

- subir un archivo incorrecto
- olvidar incluir archivos JSON
- no respetar la estructura del proyecto
- no probar el programa antes de entregar

💡 Recomendaciones

- hacer resguardos o copias antes de entregar
- probar el proyecto con `python main.py`
- respetar la estructura pedida
- leer los comentarios del profesor antes de continuar

🎯 Objetivo pedagógico

Este sistema está diseñado para:

- obligar a trabajar de forma progresiva
- validar cada etapa del proyecto
- evitar acumulación de errores
- fomentar buenas prácticas de desarrollo

📊 Evaluación

Nota final sobre 20

Criterio	Puntos
Lógica del juego	5
Modularidad (sin POO)	3
Uso de JSON	3
Interfaz terminal (ASCII)	3
Eventos y evolución	3
Calidad del código	2
Documentación	1

☑ Requisitos para aprobar

- funciones (NO clases)
- diccionarios y listas
- lectura/escritura JSON
- modularización
- bucle principal funcional
- interacción en terminal

▨ Ejemplo de documentación "Docstring" (ejemplo)

```python
def alimentar(mascota, cantidad):
    """
    Alimenta la mascota.

    Parameters:
        mascota (dict)
        cantidad (int)

    Returns:
        dict
    """
    mascota["energia"] += cantidad
    return mascota
```

💾 JSON mascota (ejemplo)

```json
{
    "nombre": "Fluffy",
    "energia": 80,
    "felicidad": 70,
    "salud": 90
}
```

🎨 ASCII Art (ejemplo)

```
_     )---(\          /~~~\
\\   (/ . . \)        /  .. \
 \\__)-\(*)/         (_,\  |_)
 \_       (_         /   \@/    /^^^\
 (___/-(____) _     /      \   / . . \
              \\   /  `    |   V\ Y /V
               \\/  \   | _\    / - \
                \   /__'|| \\_  |    \
                 \_____)|_).\_).||(__V
```

Menu ASCCI (ejemplo)

```
╔═══════════════════════════════════╗
║         🐾  TAMAGOTCHI  🐾         ╒
╟═══════════════════════════════════╢
╒ Mascota: Fluffy                   ╒
╒ Energía:  ═══════   80%           ╒
╒ Felicidad:══════    60%           ╒
╒ Salud:    ═══════   90%           ╒
╟═══════════════════════════════════╢
╒ 1. Alimentar                      ╒
╒ 2. Jugar                          ╒
╒ 3. Dormir                         ╒
╒ 4. Limpiar                        ╒
╒ 5. Inventario                     ╒
╒ 6. Guardar                        ╒
╒ 7. Salir                          ╒
╟═══════════════════════════════════╢
╒ Elige una opción:                 ╒
╚═══════════════════════════════════╝
```

🚀 Ejecución

```
python main.py
```